# Indian Institute of Technology Jodhpur

M.Tech. in Data and Computational Sciences

*Machine Learning for Big Data*

## Project Report

## Scalable Near-Duplicate Detection on Wikipedia Using Apache Spark and Locality-Sensitive Hashing


**Submitted by:**

Basu Singh (M24AID008)
Nitin Jain (M24AID022)
Shashi Saurav (M24AID048)


**GitHub Link:**
https://github.com/Yoursuperherobasu/BigData_Project_Machine_Learning-_for_Big_Data.git

## Tables of Contents

*1. Introduction* ..... *3*

*2. Literature Review* ..... *4*

*3. Problem Statement and Objectives* ..... *5*

*4. System Architecture and Design* ..... *5*

*5. Dataset Description* ..... *7*

*6. Implementation Details* ..... *8*

**6.1 Data Ingestion** ..... **8**

**6.2 Text Pre-processing and Feature Engineering** ..... **8**

**6.3 MinHash and Locality-Sensitive Hashing** ..... **9**

**6.4 Candidate Pair Generation** ..... **10**

**6.5 Evaluation Framework** ..... **10**

*7. Experimental Setup* ..... *11*

*8. Results and Analysis* ..... *11*

**8.1 Scalability Analysis** ..... **11**

**8.2 Accuracy Evaluation** ..... **12**

*9. Testing Strategy* ..... *12*

*10. How to Reproduce* ..... *13*

*11. Challenges and Lessons Learned* ..... *14*

*12. Conclusion and Future Work* ..... *16*

*13. Contribution* ..... *17*

*14. References* ..... *18*

## 13. Introduction

In the project, we chose Wikipedia for its huge number of articles ranging over six million in English. We expected each article to be different from the others but while inspecting the data, we found various articles sharing the same paragraphs and in some cases full sections. Implying that the editors were doing duplications for similar or related articles, someone copies text from one page to write another page, or two pages about similar topics end up having the same content due to it. Old pages that were merged still hang around too. All of this leads to a problem because while trying to use this data for something like building a search engine or training a classifier, the duplicate content confuses everything.

Our first idea was simple. Just go through every pair of articles and check if they match. But we quickly realised that wont work. If you have n articles then you need to check n times (n-1) divided by 2 pairs. For six million articles that's around 18 trillion pairs. Even a very fast computer cannot handle that many comparisons. On top of that, the data is too large for one machine to hold in memory.

That's why we chose to go with MinHash LSH from Spark Mllib, running on PySpark. MinHash is a technique where you hash the words of each article several times and only keep the smallest hash value each time. You get a short signature for each article this way. If two articles share many words their signatures will look alike. LSH then groups articles with similar looking signatures into the same bucket. After that you only need to compare articles that are in the same bucket, which is a much much smaller number than 18 trillion.

to test our pipeline we downloaded a shard from the March 2026 Wikipedia dump and after removing all the junk pages we had roughly 100,000 articles. Our pipeline does the following things in order: first it reads the XML, then it cleans the data, then it creates feature vectors, then it runs MinHash LSH, and finally it saves the near-duplicate pairs to disk. Furthermore, we also ran experiments to observe the variation in speed with respect to more data and whether the results are comprehensible when compared against brute force.

## 2. Literature Review

The idea of finding duplicate documents has been around since the 90s, a well renowned researcher named Broder published a paper about it in 1997 where he suggested taking each document and turning it into a set of word sequences, he defined them as shingles and further, performing comparison between the two documents by observing at how much their shingle sets overlap using something called Jaccard similarity. It's calculated by dividing the number of shared shingles by the total number of unique shingles in both sets combined. It's a very straightforward idea and it works quite well in practice.

But there is a catch. If you have a lot of documents then computing Jaccard for each and every pair takes a very long time. So Broder also invented MinHash to speed this up. He published a more detailed version of it in 1998 along with Charikar, Frieze, and Mitzenmacher. What MinHash does is it applies a random hash function to each element in the set and keeps just the smallest value. And the interesting part is that the chance of two sets producing the same smallest hash is equal to their Jaccard similarity. You do this with k different hash functions and you get k numbers as a signature for each document. Then instead of comparing full sets you just compare these short signatures.

After that Indyk and Motwani came up with something called LSH in 1998, which stands for Locality-Sensitive Hashing. It is a general method for finding approximate nearest neighbours in high dimensional spaces. The working process of it with MinHash is by dividing the signature into "b" bands of "r" rows each. Two documents are considered as a candidate pair if and only if their signatures tally perfectly in all "r" rows of at least one band. The probability of two documents with Jaccard similarity becoming candidates is represented by 1-(1-s^r)^b. Depending on what values you choose for "b" and "r" you can change the sensitivity of the filter stricter or more relaxed.

There are other researchers who have tackled similar problems on a much larger scale. Henzinger in 2006, tested different hashing methods on billions of web pages at Google. Similarly, Manku, Jain and Sarma in 2007 also proposed something called "SimHash" for finding near-duplicates in web crawls. Nowadays, Apache Spark has made it a lot easier to run similar algorithms like these by eliminating the need to build your own distributed system. Spark Mllib already has MinHashLSH as a built-in feature and it runs directly on DataFrames.

To gain the necessary understanding of the preceding theory, we focused mainly on the use of the textbook "Mining of Massive Datasets" by Leskovec, Rajaraman, and Ullman. In the book's chapter 3, MinHash and LSH are covered in detail. Our contribution in the project covers our work and how we managed to capitulate and incorporate these existing techniques and run them on real data from Wikipedia meanwhile, measuring the actual speed and accuracy numbers.

## 3. Problem Statement and Objectives

The prime focus of our project corresponds to targeting the problems regarding us having a large collection of Wikipedia articles stored as raw XML and our need to find every pair where the content is nearly identical. We defined "nearly identical" as having a Jaccard distance less than 0.3. Although this threshold can be varied, the solution must be cooperative enough to work with at least 100,000 articles, and still should take a reasonable amount of time to complete when tested on a standard spec computer consisting of multiple cores.

**Expected achievements for the project**

1. Building a pipeline on Apache Spark which will take Wikipedia XML as its input and generates a near-duplicate article pair as the expected output.
2. Using MinHash and LSH for finding candidates and avoiding the use of brute force methods for faster processing.
3. Running the pipeline on different volumes of data and recording the time taken at each stage to understand the scaling logic well.
4. Verifying accuracy by running "exact brute force Jaccard" on a small sample and then comparing it against LSH values thereby, calculating "precision","recall" and "F1".
5. Organising the code in a clean and proper format so that anyone can clone the repository and run the experiments on their own without hassle.

## 4. System Architecture and Design

Our code is categorically organised into five neat and separate python files, inside a package called "wiki_near_dup". Each file is designated to handle one part of the pipeline and "ingest.py" is used to perform the XML reading and data cleaning. "features.py" creates the vectors. Meanwhile, "lsh_pipeline.py" is used to run the MinHash algorithm and the "similarity

join". "evaluate.py" is used to calculate timing and accuracy metrics while "jaccard_utils.py" includes helper functions written in Python without Spark. The reasoning behind keeping them separate is to make it easier to debug. When something is not working as per our expectations, It gets easier to check each module separately rather than searching through one big file.

**the data flows through the pipeline in this order:**

Wikipedia XML Dump > Spark XML Parser > Article Normalisation > Tokenisation + HashingTF > Binary Feature Vectors > MinHash LSH (Fit + Transform) > Self-Join on Candidate Buckets > Jaccard Distance Filtering > Output Parquet (Near-Duplicate Pairs)

We heavily relied on the Spark DataFrames for most needs because they are easier to work with as compared to RDDs. Spark also, automatically optimises the execution plan. For parsing the Wikipedia XML, we used a library called "spark-xml". It reads the XML file and converts each page element directly into a row in the DataFrame. We avoided the mistake earlier on of not caching intermediate results. When Spark did not recompute the same data over and over again and eventually led the program to crash because it ran out of memory. The fix was simple and achieved by the introduction of "cache() calls" after "expensive transformations" and "unpersist() calls", and during the period as soon as the data was processed by the next stage. Getting this caching pattern right took a few attempts.

**Below is what each module in the codebase does:**

| Module | Responsibility |
|---|---|
| **ingest.py** | reads the Wikipedia XML with spark-xml, filters out non-articles and redirects and short pages, has a sampling function for experiments |
| **features.py** | splits text into tokens with RegexTokenizer then creates binary 0/1 vectors using HashingTF |
| **lsh_pipeline.py** | fits the MinHashLSH model on the feature vectors and runs the approximate similarity self-join to find candidate pairs |
| **evaluate.py** | handles the scaling study (timing at different data sizes) and accuracy evaluation (precision recall F1 against brute force) |
| **jaccard_utils.py** | plain Python functions for Jaccard similarity, Jaccard distance, brute force pair finding, and precision/recall/F1 calculation |

| | |
|---|---|
| **config.py** | stores default values for hyperparameters like number of hash tables, distance threshold, hash dimensions, and the spark-xml Maven coordinate |
| **run_pipeline.py** | the main command line entry point, parses arguments and calls the other modules in the right order |
| **spark_submit.sh** | a shell script that sets up the environment and passes the spark-xml package to spark-submit so you don't have to type it every time |

## 13. Dataset Description

The data we used for our experiments is one shard from the March 2026 English Wikipedia dump. The exact file name is enwiki-20260301-pages-articles-multistream2.xml-p41243p151573.bz2. it contains pages with IDs ranging from 41,243 to 151,573.

This file is about 380 MB when compressed with bzip2. When we parsed it and removed all the non-article pages, redirect pages, and articles that had less than 50 characters of actual text, we were left with roughly 100,000 usable articles. Each article has three fields: a numeric page_id, the title, and the full text of the article.

The reason we used only one shard instead of the entire Wikipedia dump is simple: the full dump would have been too large to process on our personal laptops. One shard is big enough to give meaningful experimental results. And the nice thing is our code does not assume anything about the number of shards, so if you wanted to run it on the full dump on HDFS you would just change the input path.

| Property | Value |
|---|---|
| **Source** | Wikimedia Foundation dumps (dumps.wikimedia.org) |
| **Dump date** | 20260301 (March 2026) |
| **File** | enwiki-20260301-pages-articles-multistream2.xml-p41243p151573.bz2 |
| **Compressed size** | ~380 MB |
| **Format** | MediaWiki XML inside bzip2 |
| **Page ID range** | 41,243 to 151,573 |
| **Articles after filtering** | ~100,000+ |

| Article schema | page_id (long), title (string), text (string) |
|---|---|

## 6. Implementation Details

### 6.1 Data Ingestion

The "ingest.py" file uses a library called spark-xml to read the Wikipedia XML. We configured it with rowTag set to page, parse mode set to "PERMISSIVE", and encoding set to "UTF-8". What this does is take each "<page>" element in the XML and turn it into one row of a "Spark DataFrame".

After the XML is parsed we apply several cleaning steps. We only keep articles from "namespace 0" which is the main article namespace, so all the "Talk pages", "User pages" and other special pages get removed. Redirected pages are also dropped, implying that, the pages whose text starts with the word "#REDIRECT". Any article that had fewer than 50 character limit was also removed because they were stubs and had no real content in them. After all these filtering operations, we were left with a "DataFrame" containing three columns namely, "page_id", "title" and "text".

A "sampling function" was also written which accepts a fraction and a "random seed". "seed 42" was mostly used so that iterating the same experiment gave the same sample every time. This was necessary for the scaling experiments where we wanted to make sure our results are consistent and viable for reproducibility.

### 6.2 Text Pre-processing and Feature Engineering

The "features.py" module takes input as each article's raw text and then converts it into a binary sparse vector. It was set up as a part of a two-stage, Spark ML Pipeline by us.

The first stage is a "RegexTokenizer" and its purpose is to split the text on any "non-word" character using the pattern "\W+". First, it converts everything to lowercase and then removes the tokens that are shorter than "2" characters. We also considered the addition of "stemming" and "stopword removal" but after testing and trying it out, we came up with the conclusion of not keeping them as the raw tokens already gave us good enough results for "Jaccard comparison" and the extra processing made things noticeably slower.

The second stage consists of a “HashingTF” set to binary mode with “2^18” or “262,144” dimensions. Binary mode dictates the duality at each position in the vector at any time being either “0” or “1”. It only tells us whether a particular token appeared in the article or not. It does not consider the number of occurrences or frequency. We require this because “Jaccard similarity” is meant for comparing sets, where each element is either present or absent. Our experiment dealt with various dimension sizes and we observed that using “2^16” dimensions gave us too many hash collisions which ultimately affected the accuracy. “2^20” dimensions was comparatively better than “2^18” but it exhausted more memory and the improvement was not exponential enough for its justification.

In the end, each article had a new column called “features” which contained a “SparseVector object”. These vectors are used as inputs in MinHash and LSH models.

### 6.3 MinHash and Locality-Sensitive Hashing

The file “lsh_pipeline.py” contains the main duplicate detection logic. It uses Spark Mllib’s “MinHashLSH” class. Its purpose is to look at the non-zero positions in each article’s feature vector and then apply several different hash functions to those positions. For each hash function, it keeps only the minimum value and the result comes out to be a short, numeric signature for each article.

There is a mathematical property that makes this work. For any two sets “A” and “B”, if you pick a random hash function and apply it to both sets, the probability that the minimum hash is the same for both equals their Jaccard similarity. In formula form that is “P(min h(A) = min h(B)) = |A intersect B| / |A union B|”. So if you repeat this with “k” different hash functions you get a “k-length” signature and the fraction of positions where the two signatures agree estimates the true Jaccard similarity between the two sets.

LSH takes this further by splitting each signature into multiple bands. Two articles are marked as a candidate pair if their signatures match completely within any one of these bands. The effect of this is that articles which are genuinely similar will almost certainly match in at least one band, while articles that are very different will almost never match. In our experiments we used 5 hash tables which translates to 5 bands.

## 6.4 Candidate Pair Generation

to actually generate the candidate pairs we call approxSimilarityJoin from Spark Mllib. This function performs a self-join on the DataFrame using a Jaccard distance threshold of “0.3”. In other words it returns all pairs of articles whose estimated similarity is at least “0.7”.

The self-join returns both (A,B) and (B,A) for each pair so we add a filter to keep only rows where “page_id_a” is less than “page_id_b”. This removes the duplicate entries and the final output is left with five columns namely, “page_id_a”, “title_a”, “page_id_b”, “title_b”, and “jaccardDist”. We save this as a Parquet file using Snappy compression because Parquet is fast to read and Snappy keeps the file size compact.

## 6.5 Evaluation Framework

The “evaluate.py” module supports two different type of evaluation modes:

**Scalability Study:**

In this mode the pipeline runs multiple times at different sample sizes. For example, At “1” percent, “2” percent, “5” percent, and “10” percent of the articles. For each run it records the frequency of articles in the sample and how many candidate pairs were found along with the information of wall-clock time each stage of the pipeline took. All of this data gets saved to a “JSON file” which is later used to create the plots for the report.

**Accuracy Evaluation:**

In this mode we take a small sample, usually up to 400 articles, and compute exact Jaccard distances for every single pair using brute force. This gives us the ground truth and as next step we run LSH on the exact same sample and compare its output to the ground truth. The module calculates precision, recall, F1 score, and counts of true positives, false positives, and false negatives.

The “jaccard_utils.py” file consists of all the helper functions needed for this function to compute Jaccard similarity and distance, a brute force pairwise comparison function, and a function to calculate the precision recall and F1 metrics.

## 13. Experimental Setup

| Component | Specification |
|---|---|
| **Programming Language** | Python 3.10 – 3.12 |
| **Distributed Framework** | Apache Spark 3.4 – 3.5 (PySpark) |
| **ML Library** | Spark Mllib (MinHashLSH) |
| **XML Parsing** | spark-xml (com.databricks:spark-xml_2.12:0.18.0) |
| **Java Runtime** | OpenJDK 11 / 17 |
| **Output Format** | Apache Parquet with Snappy compression |
| **Testing Framework** | pytest |
| **Version Control** | Git + GitHub |

**these are the default hyperparameter values we used across all our experiments:**

| Parameter | Default Value | Description |
|---|---|---|
| **num_features** | 2^18 (262,144) | Dimensionality of the HashingTF vector space |
| **min_token_length** | 2 | Minimum token length after tokenisation |
| **num_hash_tables** | 5 | Number of LSH hash tables (bands) |
| **jaccard_distance_threshold** | 0.3 | Maximum Jaccard distance for candidate pairs |
| **sample_seed** | 42 | Random seed for reproducible sampling |
| **max_docs_bruteforce** | 400 | Maximum documents for exact brute-force evaluation |

## 8. Results and Analysis

### 8.1 Scalability Analysis

For the scalability experiment we ran the pipeline at four different sample sizes: 1 percent, 2 percent, 5 percent, and 10 percent of the total articles. At each size we timed three stages independently: the ingestion and sampling stage, the tokenisation and feature extraction stage, and the LSH model fitting plus self-join stage.

What we observed is that the ingestion and feature extraction stages grew more or less proportionally as we added more data. This was expected because both of these stages work on one article at a time without looking at any other article. The LSH self-join stage was a different

story though. It grew faster than linear because as you add more articles the number of potential candidate pairs also goes up. But even so the growth was nowhere close to the quadratic curve that you would see if you did brute force comparisons.

Running on a laptop with multiple CPU cores, Spark was able to process a few thousand articles every minute. The biggest bottleneck in terms of speed was the "bzip2" decompression step because "bzip2" can only decompress a single file using one thread. If the data were stored on HDFS where it is already split into multiple blocks this bottleneck would not exist.

### 8.2 Accuracy Evaluation

For the accuracy test we took a random sample of up to "400" articles. In this sample we computed exact Jaccard distances between every possible pair of articles. Any pair whose distance was "0.3" or lower was considered a true "near-duplicate" and went into the ground "truth set".

Then we ran the "LSH pipeline" on the same "400 articles" using 5 hash tables and a distance threshold of "0.3". The "precision" and "recall" numbers we got were both very high in value. The small number of pairs that LSH missed were all cases where the "Jaccard distance" was around the value "0.3", which fell right at the boundary condition. This is where the LSH probability curve showed a sharp drop as well thereby, leading to some borderline pairs getting missed.

The results obtained also closely matched theoretical predictions. MinHash LSH proved to be very reliable during our testings at catching pairs that lied clearly within the similarity threshold. The errors tend to happen only at the boundary where the distance was observed to fall around the threshold values.

Brute-force ground truth found 19 near-duplicate pairs in 825-doc sample.
LSH with 5 hash tables, threshold 0.3 found exactly the same 19 pairs.

## 9. Testing Strategy

We wrote two kinds of tests for this project. The first kind runs without Spark and proved to be faster than the second kind which needed a running Spark session and took more time.

**Unit Tests (test_evaluate_pure.py):**

These tests are written in plain Python with no dependency on Spark or the JVM. We checked four things majorly including, that the Jaccard similarity of two identical sets is 1.0, that two completely different sets give 0.0, that the brute force comparison function correctly identifies pairs below a given distance threshold, and that the precision recall and F1 computation produces the correct output when given known values of true positives, false positives and false negatives respectively. These tests were recorded to take less than a second.

**Integration Tests (test_spark_smoke.py):**

This test creates a local Spark session and runs the entire pipeline on a tiny XML file that we wrote by ourselves. The file contains three articles consisting of, two of them being intentionally similar to each other and one as a redirect page. The test verifies that the XML parser reads all three pages and that the redirect gets filtered out during the cleaning phase. The LSH algorithm also finds at least one candidate pair between two similar papers. We put this test behind an environment variable called "RUN_SPARK_INTEGRATION" so that it does not run automatically during normal development.

## 10. How to Reproduce

**what you need before starting:**

Python 3.10, 3.11, or 3.12 (Python 3.13 is not recommended due to distutils compatibility issues with PySpark 3.5)
Java 11 or Java 17
Internet access on first run (Spark downloads the spark-xml JAR from Maven Central)

**Setup:**

```
git clone
https://github.com/Yoursuperherobasu/BigData_Project_Machine_Learning-
_for_Big_Data.git
cd BigData_Project_Machine_Learning-_for_Big_Data
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

**Running the pipeline:**

```
chmod +x scripts/spark_submit.sh
export SPARK_MASTER=local[*]

./scripts/spark_submit.sh \
  --input file:///absolute/path/to/enwiki-dump.xml.bz2 \
  --output file:///absolute/path/to/output_pairs \
  --sample-fraction 0.05 \
  --num-partitions 64
```

**Running tests:**

```
# Unit tests (fast, no Spark needed)
pytest tests/test_evaluate_pure.py -q

# Integration tests (requires local Spark)
export RUN_SPARK_INTEGRATION=1
pytest tests/test_spark_smoke.py -v
```

## 11. Challenges and Lessons Learned

There were several issues we ran into during development. Here are the main ones and how we handled them.

**XML Parsing:** the Wikipedia XML format is deeply nested and quite messy. The actual article text is buried inside a revision element, and depending on which dump file you use this element can show up as either a Struct type or an Array type in Spark's schema. Both cases were handled by carefully checking the schema at the runtime. Initially, we tried to use Python's built-in "ElementTree library" for parsing but it proved to not be compatible for work in the Spark's distributed execution mode, therefore, we switched to spark-xml which is designed for this case.

**Python Version Problems:** One of our team members had "Python 3.13" installed on their laptop and while trying to start a Spark session it showed very confusing "JVM classpath" errors. We spent quite some time debugging this before we discovered this mismatch and that Python 3.13 removed the distutils module which PySpark 3.5 still depends on internally. The fix was simple and it was to use Python 3.12 instead and install the setuptools package which provides a replacement for distutils.

**Memory Issues:** The first time we ran the pipeline on a large sample it crashed because Spark ran out of memory. The problem came out to be that without explicit caching Spark tries to

recompute intermediate DataFrames from scratch whenever they are needed again. We fixed this issue by introducing cache() calls after expensive transformations and unpersist() calls as soon as the data was consumed by the next stage. Getting this caching pattern right took a few attempts.

**Large File on GitHub:** The Wikipedia dump shard is around 380 MB in size but GitHub only allows files up to 100 MB. Therefore, we decided to not include the data file in the repository and instead we put the download link in the “README”, so people can get it easily by themselves.

## 12. Conclusion and Future Work

In this project we successfully built a distributed pipeline that can find near-duplicate articles in a large Wikipedia dataset. The pipeline runs on Apache Spark and uses MinHash LSH from Spark Mllib to avoid the extremely expensive brute force approach. With our pipeline we were able to process over 100,000 articles which would not be feasible if we compared every pair directly.

The experiments we ran showed that the stages which process individual articles scale linearly as you add more data. The LSH join stage grows a bit faster than linear but stays well below quadratic. The accuracy experiments showed that LSH with our default settings produces results that closely match the exact brute force ground truth, with high precision and recall values. The codebase is well structured and organized into separate modules for ease. It contains tests at two levels and also comes with documentation.

**Future Expectations:**

- **Scaling to Full Wikipedia:** We plan to run the pipeline on the complete English Wikipedia dump using a dedicated "HDFS" cluster.
- **Character-Level N-Grams:** Experiment with character-level n-gram shingles as an alternative to word tokens. This approach could be more effective at catching near-duplicates created by minor textual edits or "small tweaks" that word-based tokens might miss.
- **Hyperparameter Optimization:** A systematic search over the number of hash tables and bands is needed to identify the optimal configuration. We will plot precision-recall curves across different parameter settings to visualize these trade-offs.
- **Real-time Processing with Spark:** Integrating Spark Structured Streaming would allow us to handle new Wikipedia articles as they arrive. This ensures we can check for duplicates in real-time without the overhead of "reprocessing everything" from scratch.
- **Comparative Analysis:** To better understand the trade-offs of our current system, we should compare the MinHash LSH approach against other similarity methods, such as "SimHash" or dense embedding-based "Approximate Nearest Neighbor" (ANN) searches.

## 13. Contribution

| Name | Roll. no | Contribution |
|---|---|---|
| **Basu Singh** | **M24AID008** | Pipeline implementation (ingest, features, LSH, evaluation modules), Streamlit web UI, CLI scripts, GitHub repository setup, documentation (README, setup guides) |
| **Shashi Saurav** | **M24AID048** | Unit and integration testing and development, code review and quality assurance, dataset preparation and preprocessing validation, version management and environment setup (Python/Spark compatibility) |
| **Nitin Jain** | **M24AID022** | Project Report, literature review, system architecture and design specifications, experimental analysis, testing and result interpretation |

## 14. References


[1] Broder, A. Z. (1997). On the resemblance and containment of documents. In Proceedings of the Compression and Complexity of Sequences, pp. 21-29. IEEE.

[2] Broder, A. Z., Charikar, M., Frieze, A. M., & Mitzenmacher, M. (1998). Min-wise independent permutations. In Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC), pp. 327-336.

[3] Indyk, P., & Motwani, R. (1998). Approximate nearest neighbors: towards removing the curse of dimensionality. In Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC), pp. 604-613.

[4] Henzinger, M. (2006). Finding near-duplicate web pages: a large-scale evaluation of algorithms. In Proceedings of the 29th Annual International ACM SIGIR Conference, pp. 284-291.

[5] Manku, G. S., Jain, A., & Sarma, A. D. (2007). Detecting near-duplicates for web crawling. In Proceedings of the 16th International Conference on World Wide Web (WWW), pp. 141-150.

[6] Leskovec, J., Rajaraman, A., & Ullman, J. D. (2020). Mining of Massive Datasets. Cambridge University Press. Chapter 3: Finding Similar Items.

[7] Apache Spark Documentation. MLlib - MinHashLSH. https://spark.apache.org/docs/latest/ml-features.html#minhash-for-jaccard-distance

[8] Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., & Stoica, I. (2010). Spark: Cluster Computing with Working Sets. In Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing.

[9] Wikimedia Foundation. Wikimedia Downloads: Database Dumps. https://dumps.wikimedia.org